

Bringing Context-Aware Completion Suggestions to Arbitrary Text Entry Interfaces

Timothy J. Aveni
tja@berkeley.edu
University of California, Berkeley
Berkeley, California, USA

Armando Fox
fox@berkeley.edu
University of California, Berkeley
Berkeley, California, USA

Björn Hartmann
bjoern@berkeley.edu
University of California, Berkeley
Berkeley, California, USA

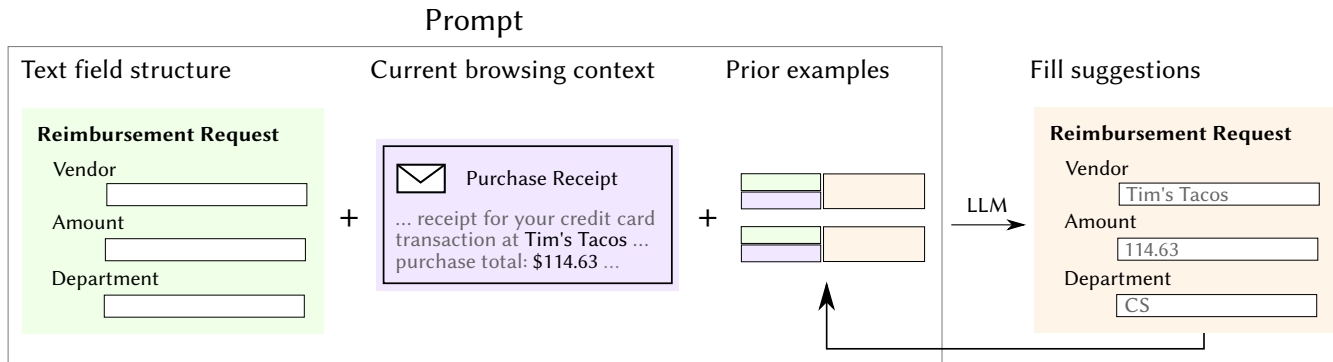


Figure 1: The structure of OmniFill’s prompt to the LLM. Suggestions are inferred from browsing context and saved examples.

ABSTRACT

Large language models (LLMs) can predict “obvious” next steps that users will take in text entry fields, especially the tedious components of tasks like software engineering or email composition. These models are not only useful in large, unbroken text fields, however. We present OmniFill, a browser extension that detects text entry fields and offers “autofill”-style suggestions based on context from the browsing session. The system constructs an LLM prompt that includes three main components: (a) a description of the active tab’s text fields and their current values, (b) information from the user’s recent web browsing context, and (c) a history, if available, of the user’s prior submissions to the web form (alongside those submissions’ associated browsing context). Suggestions from the LLM’s response are offered to the user to be automatically typed into each corresponding text field. We offer a motivating example of a time-saving interaction and discuss the broader utility of interface-agnostic LLM integrations.

CCS CONCEPTS

• **Human-centered computing** → **User interface design**; **Web-based interaction**; **Natural language interfaces**; **Text input**.

KEYWORDS

large language models, Web accessibility, intelligent user interfaces

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST '23 Adjunct, October 29–November 01, 2023, San Francisco, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0096-5/23/10.

<https://doi.org/10.1145/3586182.3615825>

ACM Reference Format:

Timothy J. Aveni, Armando Fox, and Björn Hartmann. 2023. Bringing Context-Aware Completion Suggestions to Arbitrary Text Entry Interfaces. In *The 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23 Adjunct)*, October 29–November 01, 2023, San Francisco, CA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3586182.3615825>

1 INTRODUCTION

Existing work in mixed-initiative user interfaces has enabled automated agents to observe user behavior and offer suggestions to users who are otherwise directly manipulating their interfaces [4]. Other work provides “intelligent copy-and-paste” functionality, enabling users to transport information between interfaces without needing to reformat it manually, although only certain types of information are supported [6]. Systems that enable users to automate repetitive UI tasks can improve efficiency, but many such tools require the user to specify exactly how to perform each task, for example using a scripting language [1, 5].

Large language models (LLMs) can be successful at predicting “obvious” next steps that users will take in text entry fields [2, 9]. Recent work has also highlighted their ability to extract information from user-facing interfaces [7]. By enabling an LLM to “observe” a user’s behavior as the user gathers context from various sources, we can generate potential completions and present field-level suggestions back to the user for automatic insertion. We present OmniFill, a system that offers zero-configuration interoperable intelligent suggestions across arbitrary Web interfaces.

Existing browser autofill systems can be useful for rigidly-defined tasks, like entering payment details or a physical address. However, they are not flexible enough to adapt to new types of interfaces, even when the user’s intent is readily apparent. Integrating with an LLM allows the system to make use of natural language labels on

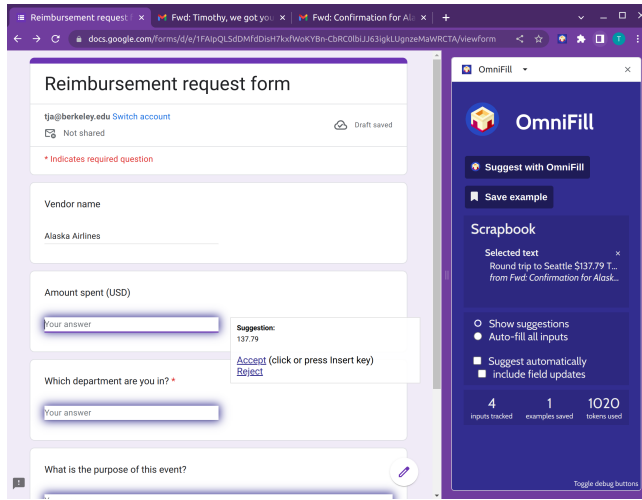


Figure 2: The OmniFill browser extension sits in a sidebar and accompanies the user as they browse the web.

text entry fields and to infer reasonable suggestions for forms with varying structures. Although some prior work ([3, 8]) has proposed algorithmic methods to locate appropriate context to fill arbitrary forms, LLMs can offer additional generative and pattern-recognition abilities to synthesize and consider all of the information available to the system.

OmniFill makes its predictions using both the current web browsing context and prior examples of the user filling out the same form. Keeping track of browsing context (including the values of the form’s fields at the time of invocation) helps the system capture the user’s current objective and information relevant to that objective. Including prior examples enables the system to learn how certain fields are formatted and to carry over information that is unlikely to change from previous interactions with the form. Even when form fields are poorly labeled, patterns in prior examples can help the system infer each field’s purpose for future invocations.

2 IMPLEMENTATION

OmniFill is implemented as a Google Chrome extension with an accompanying program that simulates mouse and keyboard inputs to fill in form fields (for maximal interoperability between websites). We chose the Web as a platform for its ease of prototyping and rich semantic information present in most text entry user interfaces. We used the gpt-3.5-turbo-16k language model, provided in an API by OpenAI, for its balance between large context size (16,384 tokens) and reasonable cost.

2.1 Prompt structure

OmniFill’s prompt to the chat-based LLM consists of a pair of “user”/“assistant” chat messages for each prior form fill example, in which the “assistant” response is in the form of a JSON object representing the fill value for each text field. The “user” side of each example, as well as the final message for the current prompt, is a JSON object containing the browsing context since the previous

query and the inferred text fields with their current values, and a brief task prompt instructing the model to fill in the JSON template.

2.1.1 Inferred text fields. OmniFill uses the DOM to locate text fields from the target UI and infers labels for each field using screen reader-accessible labels or nearby text. These field names are transformed into a JSON object mapping each field’s name to its current value.

2.1.2 Browsing context. Browsing context is collected and inserted into the prompt as the user browses the web in Google Chrome. When the user makes a Google search query, this query text is added to the prompt object. In addition, when the user selects text on any viewed page while holding the Alt (or Option) key, OmniFill collects the plain text value of the selection, as well as 500 characters of additional context before and after the selected text; only context that is manually selected in this way will be included in the prompt.

2.1.3 Model output. We call the LLM API, setting the model’s temperature to 0 and maximum response length to 1,024 tokens.

Because the prompt includes prior examples in which the “assistant” responds in the appropriate JSON format (the filled-in template object), we have not observed the model format its response incorrectly, except in cases where no examples are available (and even these are rare).

2.2 Invocation

The OmniFill sidebar offers a manual invocation button that generates a prompt using the currently active tab’s form fields and the collected browsing context. Alternatively, OmniFill can be placed into continuous invocation mode, which re-invokes the model each time new context is added to the Scrapbook or a form field is changed by the user (with a one-second debounce).

After the API call completes (which generally takes a few seconds), the response is parsed and each value in the returned JSON object is mapped to its corresponding field in the form’s browser tab. Each field for which the LLM suggested a completion different than the current value of the field is highlighted. Each field can have its suggestion individually accepted, which causes the accompanying desktop software to simulate clearing the field and typing the suggestion.

When the user is satisfied with a form’s filled values, they can press “Save example” in OmniFill’s sidebar. This stores the current browsing context and field values as an example to be used in later prompts in this browsing session, then clears the context to prepare for the next form invocation. This manual process enables users to validate the results of previous invocations before storing them for use in future prompts.

3 EXAMPLE USE CASES

3.1 Repetitive form-filling

Consider a user filling out a few expense reimbursement forms on their workplace’s accounting website, one for each event they hosted in the past month. Although this is a partially manual process, many steps can be inferred automatically. For example, some fields will be the same across multiple reimbursement forms for a particular user (like “Accounting Department”). Others will be

apparent from web browsing context; for example, an email receipt may contain the amount spent and vendor name. With OmniFill enabled, the process of using the existing reimbursement interface is streamlined:

- While locating information to be entered into the form (for example, from a credit card statement), the user uses Alt+text selection to add relevant context to the Scrapbook.
- The user fills out the first reimbursement form manually, pressing the “Save example” button in OmniFill’s sidebar before submitting the form.
- Moving onto the next form, the user locates relevant information again.
 - This new context can be formatted differently than in the previously-filled form; for example, this form’s dollar amount may come from a receipt email rather than a credit card statement.
 - The user doesn’t need to be precise in selecting this context. For example, it’s okay to add an entire email into the Scrapbook rather than just selecting the receipt total.
- Now, in the tab for the reimbursement form, the automatically-invoked suggestion engine offers potential completions to the same form fields from the first example. OmniFill can make inferences based on prior examples (e.g. offering the same completion as the previous form for unchanging fields, or correctly formatting a date or dollar amount) or based on the current browsing context. The user can accept or reject suggestions individually for each form field.
- As more examples are added, OmniFill can more easily suggest completions that follow patterns in previous examples.

We have trialed this workflow in a mocked-up reimbursement form created in Google Forms, and OmniFill is surprisingly adept at offering accurate suggestions for form fields. In the few situations we tested in which the system provided poor suggestions, saving additional examples for the form consistently improved outputs. We have found that in many cases, the LLM prefers to leave a field blank rather than hallucinating an incorrect suggestion, even without including explicit instructions effect in the prompt. In cases where the system’s suggestions are incorrect, users can “reject” the suggestions to remove them from view, or simply ignore them. In the automatic invocation mode, suggestions can become more relevant once the user has filled out more of the form, since the system will re-prompt the LLM with this updated context.

3.2 Additional interfaces

Because OmniFill can identify arbitrary text entry interfaces on the Web, integration extends to many tasks with no additional configuration. Our video figure showcases additional use cases for OmniFill, such as calendar scheduling, budget management, flash card creation, and automatically filling search bars. Because LLMs are capable of fluent text generation, text entry interfaces can even function as prompt skeletons for more demanding tasks.

4 CONCLUSION

OmniFill allows users to bypass tedious portions of Web-based text entry tasks that are apparent from interaction history and browsing context. This is possible across arbitrary context sources

and target UIs, rather than locking users into individual closed software ecosystems, and we hope to extend this functionality in the future to interactions outside the Web browser.

By automatically suggesting “obvious” completions in typical user-facing input interfaces, we hope to bridge the gap between fully manual interface control and automations that would require the user to find or write code to selectively automate their workflows. Especially as LLMs become more practical to run locally and privately, we anticipate an increase in AI integration not just into tasks with explicit AI functionality, but in arbitrary tasks spanning across individuals’ computer usage.

REFERENCES

- [1] Allen Cypher. 2012. Automating data entry for end users. In *2012 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 23–30. <https://doi.org/10.1109/VLHCC.2012.6344474>
- [2] Steven M. Goodman, Erin Buehler, Patrick Clary, Andy Coenen, Aaron Donsbach, Tiffanie N. Horne, Michal Lahav, Robert MacDonald, Rain Breaw Michaels, Ajit Narayanan, Mahima Pushkarna, Joel Riley, Alex Santana, Lei Shi, Rachel Sweeney, Phil Weaver, Ann Yuan, and Meredith Ringel Morris. 2022. LaMPPost: Design and Evaluation of an AI-Assisted Email Writing Prototype for Adults with Dyslexia. In *Proceedings of the 24th International ACM SIGACCESS Conference on Computers and Accessibility (Athens, Greece) (ASSETS '22)*. Association for Computing Machinery, New York, NY, USA, Article 24, 18 pages. <https://doi.org/10.1145/3517428.3544819>
- [3] Melanie Hartmann and Max Muhlhauser. 2009. Context-Aware Form Filling for Web Applications. In *2009 IEEE International Conference on Semantic Computing*. 221–228. <https://doi.org/10.1109/ICSC.2009.83>
- [4] Eric Horvitz. 1999. Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI conference on Human Factors in Computing Systems*. 159–166.
- [5] Chris Mallett and Steve Gray. 2023. AutoHotkey. <https://www.autohotkey.com/>
- [6] Jeffrey Stylos, Brad A. Myers, and Andrew Faulring. 2004. Citrine: Providing Intelligent Copy-and-Paste. In *Proceedings of the 17th Annual ACM Symposium on User Interface Software and Technology (Santa Fe, NM, USA) (UIST '04)*. Association for Computing Machinery, New York, NY, USA, 185–188. <https://doi.org/10.1145/1029632.1029665>
- [7] Dakuo Wang, Elizabeth Churchill, Pattie Maes, Xiangmin Fan, Ben Shneiderman, Yuanchun Shi, and Qianying Wang. 2020. From Human-Human Collaboration to Human-AI Collaboration: Designing AI Systems That Can Work Together with People. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems (Honolulu, HI, USA) (CHI EA '20)*. Association for Computing Machinery, New York, NY, USA, 1–6. <https://doi.org/10.1145/3334480.3381069>
- [8] Shaohua Wang, Ying Zou, Bipin Upadhyaya, and Joanna Ng. 2013. An Intelligent Framework for Auto-filling Web Forms from Different Web Applications. In *2013 IEEE Ninth World Congress on Services*. 175–179. <https://doi.org/10.1109/SERVICES.2013.19>
- [9] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity Assessment of Neural Code Completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming (San Diego, CA, USA) (MAPS 2022)*. Association for Computing Machinery, New York, NY, USA, 21–29. <https://doi.org/10.1145/3520312.3534864>